Object–oriented programming

Instructor: Masoud Asghari

Web page: http://www.masses.ir/lectures/oops2017sut

Ch: 2

In this slide

- We follow:
  - https://docs.oracle.com/javase/tutorial/index.html

Trails Covering the Basics

These trails are available in book form as *The Java Tutorial, Sixth Edition*. To buy this book, refer to the box to the right.

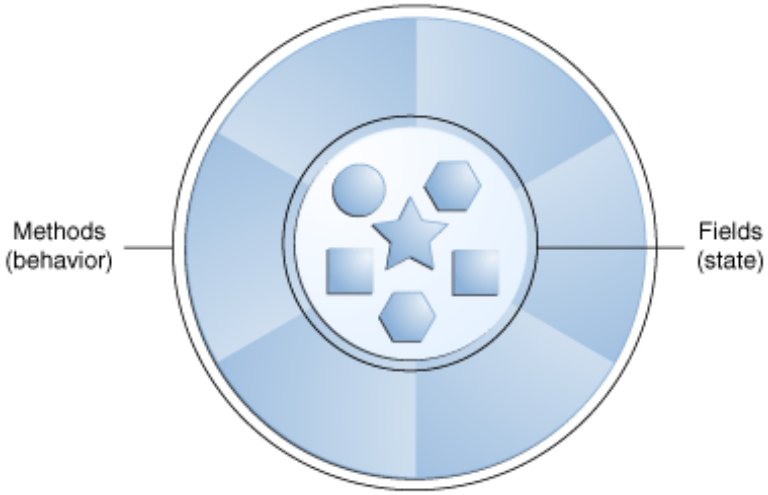
- Getting Started — An introduction to Java technology and lessons on installing Java development software and using it to create a simple program.
- Learning the Java Language — Lessons describing the essential concepts and features of the Java Programming Language.
- Essential Java Classes — Lessons on exceptions, basic input/output, concurrency, regular expressions, and the platform environment.
- Collections — Lessons on using and extending the Java Collections Framework.
- Date-Time APIs — How to use the `java.time` pages to write date and time code.
- Deployment — How to package applications and applets using JAR files, and deploy them using Java Web Start and Java Plug-in.
- Preparation for Java Programming Language Certification — List of available training and tutorial resources.

Learning the Java Language> Object-Oriented Programming Concepts> What Is an Object?

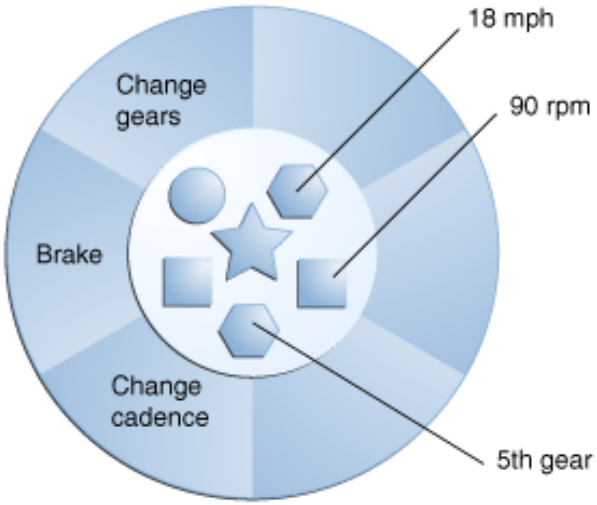
- You will be introduced to objects, classes, inheritance, interfaces, and packages.
- **What Is an Object?**
  - Objects are key to understanding object-oriented technology.
  - Look around right now and you'll find many examples of real-world objects: your *dog*, your *desk*, your *television set*, your *bicycle*.
  - **Real-world objects share two characteristics:**
    - They all have **state** and **behavior**.
    - *Dogs* have state (name, color, age, hungry) and behavior (barking, fetching, wagging tail).
    - *Bicycles* have state (current gear, current speed) and behavior (changing gear, applying brakes).
  - For each object that you see, ask yourself two questions:
    - "What possible **states** can this object be in?"
    - and "What possible **behavior** can this object perform?".
  - You may also notice that some objects, in turn, will also contain other objects.
  - These real-world observations all translate into the world of object-oriented programming.

Learning the Java Language> Object-Oriented Programming Concepts> What Is an Object?

- Software objects are similar to real-world objects:
  - they too consist of **state** and related **behavior**.
- An object
  - stores its **state** in **fields** (**variables**)
  - exposes its **behavior** through **methods** (**functions**).
    - Methods operate on an object's internal state and serve as the primary mechanism for object-to-object communication.
- Hiding internal state and requiring all interaction to be performed through an object's methods is known as **data encapsulation**



Learning the Java Language> Object-Oriented Programming Concepts> What Is an Object?



- A bicycle modeled as a software object.
  - By attributing state (current speed, current gear) and providing methods for changing that state,
    - The object remains in control of how the outside world is allowed to use it.
    - For example, if the bicycle only has 6 gears, a method to change gears could reject any value that is less than 1 or greater than 6.
- Bundling code into individual software objects provides a number of benefits, including:
  - **Modularity**: The source code for an object can be written and maintained independently of the source code for other objects. Once created, an object can be easily passed around inside the system.
  - **Information-hiding:** By interacting only with an object's methods, the details of its internal implementation remain hidden from the outside world.
  - **Code re-use:** You can use other's object in your program.
  - **Pluggability and debugging ease:** If a particular object turns out to be problematic, you can simply remove it from your application and plug in a different object as its replacement..

- **What Is a Class?**
- In the real world,
  - you'll often find many individual objects all of the same kind.
  - There may be thousands of other bicycles in existence, all of the same make and model.
  - Each bicycle was built from the same set of **blue prints** and therefore contains the same components.
- In object-oriented terms,
  - we say that your bicycle is an instance of the class of objects known as bicycles.
- A class is the blueprint from which individual objects are created.

- **Example :**
- The fields speed, and gear represent the object's state,
- and the methods (changeCadence, changeGear, speedUp etc.) define its interaction with the outside world.

- Note that the Bicycle class does not contain a main method.
  - That's because it's not a complete application; it's just the blueprint for bicycles that might be used in an application.
  - The responsibility of **creating** and using **new** Bicycle objects belongs to some other class in your application.

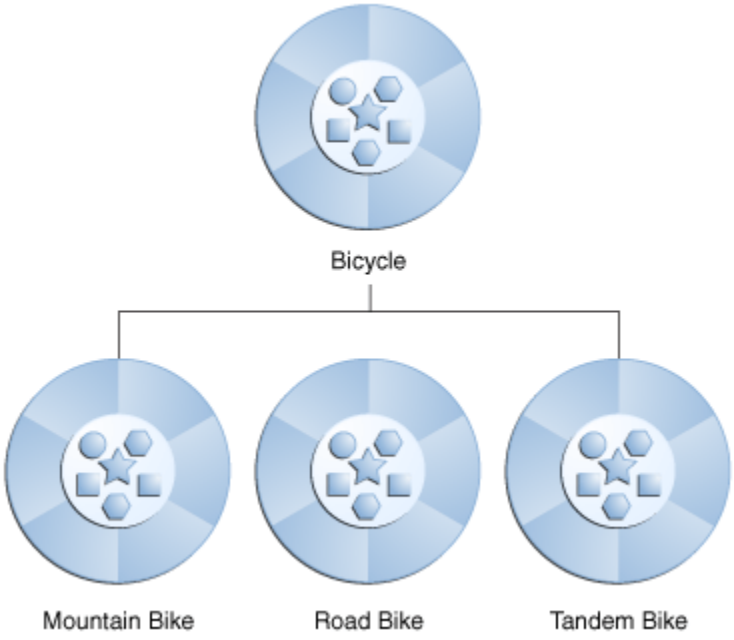
```
class Bicycle {

    int speed = 0;
    int gear = 1;

    void changeGear(int newValue) {
        gear = newValue;
    }

    void speedUp(int increment) {
        speed = speed + increment;
    }

    void applyBrakes(int decrement) {
        speed = speed - decrement;
    }

    void printStates() {
        System.out.println(" speed:" +
            speed + " gear:" + gear);
    }
}
```

- **Example:**

```java
class BicycleDemo {
    public static void main(String[] args) {

        Bicycle bike1 = new Bicycle();
        Bicycle bike2 = new Bicycle();

        bike1.speedUp(10);
        bike1.changeGear(2);
        bike1.printStates();

        bike2.speedUp(10);
        bike2.changeGear(2);
        bike2.speedUp(10);
        bike2.changeGear(3);
        bike2.printStates();
    }
}
```

- a BicycleDemo class that creates two separate Bicycle objects and invokes their method.
  - The output of this test prints the ending speed, and gear for the two bicycles:
    - speed:10 gear:2
      speed:20 gear:3

- **What Is Inheritance?**
- Different kinds of objects often have a certain amount in common with each other.
  - for example: Mountain bikes, road bikes, and tandem bikes, all **share** the characteristics of bicycles (current speed, current pedal cadence, current gear).
  - Yet each also defines additional features that make them different:
    - tandem bicycles have two seats and two sets of handlebars;
    - road bikes have drop handlebars;
    - some mountain bikes have an additional chain ring
- Object-oriented programming allows classes to **inherit** commonly used state and behavior from other classes.
  - In this example,
    Bicycle now becomes the superclass of MountainBike, RoadBike, and TandemBike.



Learning the Java Language> Object-Oriented Programming Concepts> What Is Inheritance?

- In the Java programming language,
  - each class is allowed to have one direct superclass, and each superclass has the potential for an unlimited number of subclasses
- The syntax for creating a subclass:
  - At the beginning of your class declaration, use the extends keyword, followed by the name of the class to inherit from
  - Example:

```
class MountainBike extends Bicycle {

    // new fields and methods defining
    // a mountain bike would go here

}
```

  - This gives MountainBike all the same fields and methods as Bicycle, yet allows its code to focus exclusively on the features that make it unique.

- **What Is an Interface?**
  - objects define their interaction with the outside world through the methods that they expose. Methods form the object's interface with the outside world;
  - for example: the buttons on the front of your television set, are the interface between you and the electrical wiring on the other side of its plastic casing. You press the "power" button to turn the television on and off.
- an interface is a group of related methods with empty bodies.
  - Example: A bicycle's behavior, if specified as an interface, might appear as:

```
interface Bicycle {

    void changeGear(int newValue);

    void speedUp(int increment);

    void applyBrakes(int decrement);
}
```

- To implement this interface,
  - the name of your class would change (to a particular brand of bicycle, for example, such as ACMEBicycle),
  - and you'd **use the implements keyword in the class declaration:**

```
class ACMEBicycle implements Bicycle {

    int speed = 0;
    int gear = 1;

    void changeGear(int newValue) {
        gear = newValue;
    }

    void speedUp(int increment) {
        speed = speed + increment;
    }

    void applyBrakes(int decrement) {
        speed = speed - decrement;
    }

    void printStates() {
        System.out.println(" speed:" +
            speed + " gear:" + gear);
    }
}
```

- Implementing an interface
  - allows a class to become more formal about the behavior it promises to provide.
  - **Interfaces** form a **contract** between the class and the outside world, and this contract is enforced at build time by the **compiler**.
  - If your class claims to implement an interface, all methods defined by that interface must appear in its source code before the class will successfully compile.

Note: To actually compile the ACMEBicycle class, you'll need to add the public keyword to the beginning of the implemented interface methods.

- **What Is a Package?**
  - A package is a namespace that organizes a set of related classes and interfaces.
    - you can think of packages as being similar to different folders on your computer.
- The Java platform provides an enormous class library
  - (a set of packages) suitable for use in your own applications.
  - This library is known as the "Application Programming Interface", or "**API**".
    - For example, a String object contains state and behavior for character strings;
    - a File object allows a programmer to easily create, delete, inspect, compare, or modify a file on the filesystem;
    - a Socket object allows for the creation and use of network sockets;
    - various GUI objects control buttons and checkboxes and anything else related to graphical user interfaces.
    - There are thousands of classes to choose from.
  - The *Java Platform API Specification* at https://docs.oracle.com/javase/8/docs/api/index.html
    - contains the complete listing for all packages, interfaces, classes, fields, and methods supplied by the Java SE platform.